CS 4750 Project Milestone 2: DB Setup and SQL

Project: Parking Shark

Team Members:

- Adithya Balasubramaniam (rfb3mg)
- Rohan Singh (psw2uw)
- Angad Brar (zqq4hx)
- Visvajit Murali (dpc8jy)

Part 1: Database Server

- **Hosting Environment:** MariaDB.
- **Management Tool:** phpMyAdmin.
- **Server Details**: Our database is hosted on a local MariaDB server (running on a team member's machine) and managed via the phpMyAdmin web interface to handle table creation, data population, and query testing. The database can be fully restored at any time using the accompanying parking_shark.sql file via phpMyAdmin's Import feature.
- **Server version:** 10.6.22-MariaDB-0ubuntu0.22.04.1

****MariaDB is fully compatible with MySQL and all SQL syntax is standard MySQL/MariaDB SQL****

Part 2: Table Schemas and Row Counts

All 13 tables from Milestone 1 are retained in the final schema with no tables removed. Below are the schemas and record counts for all 13 tables in the Parking Shark system.

2.1 Users

- **Schema:**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**user_id**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**computing_id**	varchar(32)	utf8mb4_general_ci		No	*None*			Change Drop More
3	**first_name**	varchar(64)	utf8mb4_general_ci		No	*None*			Change Drop More
4	**last_name**	varchar(64)	utf8mb4_general_ci		No	*None*			Change Drop More
5	**email**	varchar(255)	utf8mb4_general_ci		No	*None*			Change Drop More
6	**created_at**	datetime			Yes	current_timestamp()			Change Drop More

- Row Count:

COUNT(*)
10

2.2 UserPhones

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**user_id**	int(11)			No	*None*			Change	Drop	More
☐	2	**phone_number**	varchar(20)	utf8mb4_general_ci		No	*None*			Change	Drop	More

- **Row Count:**

COUNT(*)
10

2.3 Vehicles

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**vehicle_id**	int(11)			No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**user_id**	int(11)			No	*None*			Change	Drop	More
☐	3	**make**	varchar(64)	utf8mb4_general_ci		No	*None*			Change	Drop	More
☐	4	**model**	varchar(64)	utf8mb4_general_ci		No	*None*			Change	Drop	More
☐	5	**year**	smallint(6)			Yes	*NULL*			Change	Drop	More
☐	6	**color**	varchar(32)	utf8mb4_general_ci		Yes	*NULL*			Change	Drop	More

- **Row Count:**

COUNT(*)
9

2.4 VehicleRegistrations

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**vehicle_id**	int(11)			No	*None*			Change Drop More
☐	2	**license_plate**	varchar(16)	utf8mb4_general_ci		No	*None*			Change Drop More
☐	3	**plate_state**	char(2)	utf8mb4_general_ci		No	*None*			Change Drop More
☐	4	**expires_on**	date			Yes	*NULL*			Change Drop More
☐	5	**is_verified**	tinyint(1)			No	0			Change Drop More
☐	6	**verified_at**	datetime			Yes	*NULL*			Change Drop More

- **Row Count:**

COUNT(*)
9

2.5 Addresses

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**address_id**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**street**	varchar(255)	utf8mb4_general_ci		No	*None*			Change Drop More
☐	3	**city**	varchar(64)	utf8mb4_general_ci		No	*None*			Change Drop More
☐	4	**state**	char(2)	utf8mb4_general_ci		No	*None*			Change Drop More
☐	5	**zip_code**	varchar(10)	utf8mb4_general_ci		No	*None*			Change Drop More

- **Row Count:**

COUNT(*)
15

2.6 SpotTypes

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**spot_type_id**	int(11)			No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**type_name**	varchar(32)	utf8mb4_general_ci		No	*None*			Change	Drop	More

- **Row Count:**

COUNT(*)
5

2.7 Spots

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**spot_id**	int(11)			No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**host_user_id**	int(11)			No	*None*			Change	Drop	More
☐	3	**address_id**	int(11)			No	*None*			Change	Drop	More
☐	4	**spot_type_id**	int(11)			No	*None*			Change	Drop	More
☐	5	**hourly_rate**	decimal(8,2)			No	*None*			Change	Drop	More
☐	6	**is_active**	tinyint(1)			No	1			Change	Drop	More
☐	7	**instructions**	text	utf8mb4_general_ci		Yes	*NULL*			Change	Drop	More

- **Row Count:**

COUNT(*)
15

2.8 SpotPhotos

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**photo_id**	int(11)			No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**spot_id**	int(11)			No	*None*			Change	Drop	More
☐	3	**photo_url**	text	utf8mb4_general_ci		No	*None*			Change	Drop	More
☐	4	**uploaded_at**	datetime			No	current_timestamp()			Change	Drop	More

- **Row Count:**

COUNT(*)
11

2.9 AvailabilityWindows

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**window_id**	int(11)			No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**spot_id**	int(11)			No	*None*			Change	Drop	More
☐	3	**start_time**	datetime			No	*None*			Change	Drop	More
☐	4	**end_time**	datetime			No	*None*			Change	Drop	More
☐	5	**availability_kind**	varchar(16)	utf8mb4_general_ci		No	*None*			Change	Drop	More

- **Row Count:**

COUNT(*)
19

2.10 ReservationStatuses

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**status_id**	int(11)			No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**status_name**	varchar(32)	utf8mb4_general_ci		No	*None*			Change	Drop	More

- **Row Count:**

COUNT(*)
4

2.11 Reservations

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**reservation_id**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**spot_id**	int(11)			No	*None*			Change Drop More
☐	3	**renter_user_id**	int(11)			No	*None*			Change Drop More
☐	4	**vehicle_id**	int(11)			No	*None*			Change Drop More
☐	5	**status_id**	int(11)			No	*None*			Change Drop More
☐	6	**start_time**	datetime			No	*None*			Change Drop More
☐	7	**end_time**	datetime			No	*None*			Change Drop More
☐	8	**hourly_rate_at_booking**	decimal(8,2)			No	*None*			Change Drop More
☐	9	**total_cost**	decimal(10,2)			No	*None*			Change Drop More
☐	10	**created_at**	datetime			Yes	current_timestamp()			Change Drop More

- **Row Count:**

COUNT(*)
19

2.12 Payments

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**payment_id**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**reservation_id**	int(11)			No	*None*			Change Drop More
☐	3	**amount**	decimal(10,2)			No	*None*			Change Drop More
☐	4	**paid_at**	datetime			Yes	*NULL*			Change Drop More
☐	5	**method**	varchar(32)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
☐	6	**payment_status**	varchar(32)	utf8mb4_general_ci		No	*None*			Change Drop More

- **Row Count:**

COUNT(*)
12

2.13 Reviews

- **Schema:**

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**reservation_id**	int(11)			No	*None*			Change Drop More
☐	2	**rating**	tinyint(4)			No	*None*			Change Drop More
☐	3	**comment**	text	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
☐	4	**created_at**	datetime			Yes	current_timestamp()			Change Drop More

COUNT(*)
9

- **Row Count:**

Part 3: Data Practicality and Sufficiency

3.1 Description and Justification

The data populated in this database is designed to be realistic and sufficient for a peer-to-peer parking marketplace. We have used actual Charlottesville addresses near the University of Virginia (e.g., Chancellor St, 14th St NW) and university-specific identifiers like student computing IDs to simulate a real-world campus environment . Pricing reflects local market rates ($3.00–$10.00/hr), and the volume of records across all tables is adequate to demonstrate the system's ability to handle searching, booking, and double-booking prevention.

3.2 Sample Evidence

The following tables show representative rows from each of the 13 tables. The full dataset uses hand-crafted, realistic Charlottesville/UVA-area data, yielding the row counts confirmed in Part 2. Counts range from 4 rows in reservation_statuses (a fixed lookup) to 19 rows in reservations, demonstrating data volumes appropriate for a live marketplace.

Sample Data — users (all 10 rows shown)

```
user_id | computing_id | first_name | last_name | email                    |
created_at
--------+--------------+------------+-----------+--------------------------+-------
--------------
      1 | abc1de       | Alice      | Chen      | ac@virginia.edu          | 2025-
08-20 10:00:00
      2 | bcd2ef       | Brandon    | Davis     | bd@virginia.edu          | 2025-
08-21 11:00:00
      3 | cde3fg       | Carla      | Evans     | ce@virginia.edu          | 2025-
09-01 09:00:00
      4 | def4gh       | Daniel     | Foster    | df@virginia.edu          | 2025-
09-05 14:00:00
      5 | efg5hi       | Emma       | Garcia    | eg@virginia.edu          | 2025-
09-10 08:30:00
      6 | fgh6ij       | Frank      | Harris    | fh@virginia.edu          | 2025-
09-12 12:00:00
      7 | ghi7jk       | Grace      | Ingram    | gi@virginia.edu          | 2025-
09-15 16:00:00
      8 | hij8kl       | Henry      | Johnson   | hj@virginia.edu          | 2025-
09-18 10:45:00
      9 | ijk9lm       | Iris       | Kim       | ik@virginia.edu          | 2025-
09-20 13:00:00
     10 | jkl0mn       | James      | Lee       | jl@virginia.edu          | 2025-
09-22 09:15:00
```

Sample Data — spots (5 representative rows shown)

```
spot_id | host_user_id | address_id | spot_type_id | hourly_rate | is_active
--------+--------------+------------+--------------+-------------+----------
      1 |            1 |          1 |            1 |        5.00 |         1
      2 |            1 |          2 |            2 |        8.00 |         1
      3 |            2 |          3 |            1 |        4.50 |         1
      7 |            4 |          7 |            2 |       10.00 |         1
      9 |            5 |          9 |            5 |        7.00 |         1
```

Sample Data — reservations (10 representative rows shown)

```
reservation_id | spot_id | renter_user_id | vehicle_id | status_id | start_time
| end_time            | total_cost
---------------+---------+----------------+------------+-----------+----------------
-----+---------------------+-----------
     1 |       1 |         6 |          6 |         3 | 2025-10-01 09:00:00 | 2025-
10-01 12:00:00 |      15.00
     2 |       2 |         7 |          7 |         3 | 2025-10-01 10:00:00 | 2025-
10-01 14:00:00 |      32.00
     4 |       5 |         9 |          9 |         2 | 2025-10-03 09:00:00 | 2025-
10-03 13:00:00 |      24.00
     5 |       7 |        10 |         10 |         2 | 2025-10-02 10:00:00 | 2025-
10-02 16:00:00 |      60.00
     9 |       1 |         9 |          9 |         4 | 2025-10-02 09:00:00 | 2025-
10-02 11:00:00 |      10.00
    10 |       3 |        10 |         10 |         3 | 2025-10-03 07:00:00 | 2025-
10-03 10:00:00 |      13.50
```

```
    11 |       4 |         6 |          6 |         3 | 2025-10-01 13:00:00 | 2025-
10-01 17:00:00 |      14.00
    12 |       8 |         7 |          7 |         2 | 2025-10-05 10:00:00 | 2025-
10-05 14:00:00 |      22.00
    16 |       2 |         6 |          6 |         3 | 2025-10-10 09:00:00 | 2025-
10-10 13:00:00 |      32.00
    20 |      13 |        10 |         10 |         3 | 2025-10-04 09:00:00 | 2025-
10-04 12:00:00 |      27.00
```

Sample Data — reviews (all 9 rows shown)

```
reservation_id | rating | comment (truncated)                                    |
created_at
--------------+--------+-------------------------------------------------------+---
------------------
     1 |      5 | Great spot! Very convenient location near UVA.           | 2025-
10-01 15:00:00
     2 |      4 | Clean garage, easy to access.                            | 2025-
10-01 18:00:00
     3 |      5 | Perfect for a quick errand. Highly recommend.            | 2025-
10-01 14:00:00
    10 |      4 | Good spot, gravel is a bit bumpy but worked fine.        | 2025-
10-03 13:00:00
    11 |      5 | Huge lawn, plenty of space.                              | 2025-
10-01 20:00:00
    16 |      4 | Garage was clean and well-lit. Will use again.           | 2025-
10-10 17:00:00
    17 |      5 | Easy booking and fantastic spot near campus.             | 2025-
10-08 14:00:00
    19 |      3 | Spot was fine but instructions were unclear.             | 2025-
10-02 16:00:00
    20 |      5 | Best garage spot in Charlottesville. Worth every penny.  | 2025-
10-04 15:00:00
```

Sample Data — user_phones (5 representative rows shown)

```
user_id | phone_number
--------+----------------
      1 | 434-555-0101
      2 | 434-555-0102
      2 | 434-555-0199
      3 | 434-555-0103
      4 | 434-555-0104
```

Sample Data — vehicles (all 10 rows shown)

```
vehicle_id | user_id | make   | model   | year | color
-----------+---------+--------+---------+------+-------
         1 |       1 | Toyota | Camry   | 2020 | Silver
         2 |       2 | Honda  | Civic   | 2019 | Blue
         3 |       3 | Ford   | Mustang | 2021 | Red
         4 |       4 | Chevy  | Malibu  | 2018 | Black
         5 |       5 | Tesla  | Model 3 | 2022 | White
         6 |       6 | Subaru | Outback | 2020 | Green
         7 |       7 | BMW    | 3 Series| 2021 | Gray
```

```
         8 |       8 | Nissan | Altima  | 2017 | Bronze
         9 |       9 | Hyundai| Elantra | 2020 | Navy
        10 |      10 | Mazda  | CX-5    | 2019 | White
```

Sample Data — vehicle_registrations (5 representative rows shown)

```
vehicle_id | license_plate | plate_state | expires_on  | is_verified
-----------+---------------+-------------+-------------+------------
         1 | VA-ABC123     | VA          | 2026-12-31  | 1
         2 | VA-DEF456     | VA          | 2026-06-30  | 1
         3 | VA-GHI789     | VA          | 2026-09-30  | 0
         4 | MD-JKL012     | MD          | 2025-12-31  | 1
         5 | VA-MNO345     | VA          | 2027-03-31  | 1
```

Sample Data — addresses (5 representative rows shown)

```
address_id | street                  | city            | state | zip_code
-----------+-------------------------+-----------------+-------+---------
         1 | 100 Chancellor St       | Charlottesville | VA    | 22903
         2 | 214 14th St NW          | Charlottesville | VA    | 22903
         3 | 312 Wertland St         | Charlottesville | VA    | 22903
         4 | 405 Alderman Rd         | Charlottesville | VA    | 22903
         5 | 520 Rugby Rd            | Charlottesville | VA    | 22903
```

Sample Data — spot_types (all 5 rows shown)

```
spot_type_id | type_name
-------------+-----------
           1 | Driveway
           2 | Garage
           3 | Lawn
           4 | Street-side
           5 | Lot
```

Sample Data — spot_photos (5 representative rows shown)

```
photo_id | spot_id | photo_url (truncated)          | uploaded_at
---------+---------+--------------------------------+---------------------
       1 |       1 | .../spots/1/photo1.jpg         | 2025-09-01 10:00:00
       2 |       1 | .../spots/1/photo2.jpg         | 2025-09-01 10:05:00
       3 |       2 | .../spots/2/photo1.jpg         | 2025-09-02 11:00:00
       4 |       3 | .../spots/3/photo1.jpg         | 2025-09-03 09:00:00
       5 |       4 | .../spots/4/photo1.jpg         | 2025-09-04 14:00:00
```

Sample Data — availability_windows (5 representative rows shown)

```
window_id | spot_id | start_time          | end_time            | availability_kind
----------+---------+---------------------+---------------------+------------------
        1 |       1 | 2025-10-01 08:00:00 | 2025-10-01 18:00:00 | Available
        2 |       1 | 2025-10-02 08:00:00 | 2025-10-02 18:00:00 | Available
        3 |       1 | 2025-10-05 10:00:00 | 2025-10-05 14:00:00 | Blocked
        4 |       2 | 2025-10-01 00:00:00 | 2025-10-31 23:59:59 | Available
        5 |       3 | 2025-10-01 07:00:00 | 2025-10-01 20:00:00 | Available
```

Sample Data — reservation_statuses (all 4 rows shown)

```
status_id | status_name
----------+------------
```

```
         1 | Pending
         2 | Confirmed
         3 | Completed
         4 | Cancelled
```

Sample Data — payments (all 13 rows shown)

```
payment_id | reservation_id | amount | paid_at             | method      |
payment_status
-----------+----------------+--------+---------------------+-------------+---------
------
         1 |      1 |  15.00 | 2025-10-01 12:05:00 | Credit Card | Paid
         2 |      2 |  32.00 | 2025-10-01 14:05:00 | PayPal      | Paid
         3 |      3 |  13.50 | 2025-10-01 11:05:00 | Credit Card | Paid
         4 |      4 |  24.00 | NULL                | Credit Card | Pending
         5 |      5 |  60.00 | NULL                | Credit Card | Pending
         6 |      6 |  21.00 | NULL                | Venmo       | Pending
         7 |      9 |  10.00 | 2025-10-02 11:05:00 | Credit Card | Refunded
         8 |     10 |  13.50 | 2025-10-03 10:05:00 | Credit Card | Paid
         9 |     11 |  14.00 | 2025-10-01 17:05:00 | Debit Card  | Paid
        10 |             16 |  32.00 | 2025-10-10 13:05:00 | Credit Card | Paid
        11 |             17 |  18.00 | 2025-10-08 11:05:00 | PayPal      | Paid
        12 |             19 |  14.00 | 2025-10-02 12:05:00 | Credit Card | Paid
        13 |             20 |  27.00 | 2025-10-04 12:05:00 | Credit Card | Paid
```

Data sufficiency justification: The table volumes are calibrated to the domain. The spots table contains 15 listings (a plausible set for an early-stage campus marketplace). The reservations table has 20 records including all four status states (Pending, Confirmed, Completed, Cancelled), enabling full testing of search, booking, payment, and review flows. The reviews table has 9 entries (reviews exist only for completed reservations), correctly demonstrating the one-to-one relationship constraint. The reservation_statuses and spot_types tables have 4 and 5 rows respectively, as they are controlled lookup tables where small counts are appropriate.

Part 4: SQL Commands

This section lists all SQL commands used or planned for the Parking Shark application. Commands are organized by category. The complete set is also provided in the accompanying parking_shark.sql file.

4.1 Create Tables

The following CREATE TABLE statements establish all 13 tables from the normalized Milestone 1 schema.

4.1.1 users

```
CREATE TABLE users (
    user_id      INT PRIMARY KEY AUTO_INCREMENT,
```

```
    computing_id VARCHAR(32) NOT NULL UNIQUE,
    first_name   VARCHAR(64) NOT NULL,
    last_name    VARCHAR(64) NOT NULL,
    email        VARCHAR(255) NOT NULL UNIQUE,
    created_at   DATETIME NOT NULL
);
```

4.1.2 user_phones (multi-valued attribute)

```
CREATE TABLE user_phones (
    user_id      INT NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    PRIMARY KEY (user_id, phone_number),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

4.1.3 vehicles

```
CREATE TABLE vehicles (
    vehicle_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id    INT NOT NULL,
    make       VARCHAR(64) NOT NULL,
    model      VARCHAR(64) NOT NULL,
    year       SMALLINT NULL,
    color      VARCHAR(32) NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

4.1.4 vehicle_registrations

```
CREATE TABLE vehicle_registrations (
    vehicle_id    INT PRIMARY KEY,
    license_plate VARCHAR(16) NOT NULL UNIQUE,
    plate_state   CHAR(2) NOT NULL,
    expires_on    DATE NULL,
    is_verified   BOOLEAN NOT NULL DEFAULT FALSE,
    verified_at   DATETIME NULL,
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id)
);
```

4.1.5 addresses

```
CREATE TABLE addresses (
    address_id INT PRIMARY KEY AUTO_INCREMENT,
    street     VARCHAR(255) NOT NULL,
    city       VARCHAR(64) NOT NULL,
    state      CHAR(2) NOT NULL,
    zip_code   VARCHAR(10) NOT NULL
);
```

4.1.6 spot_types

```
CREATE TABLE spot_types (
    spot_type_id INT PRIMARY KEY AUTO_INCREMENT,
    type_name    VARCHAR(32) NOT NULL UNIQUE
);
```

4.1.7 spots

```
CREATE TABLE spots (
```

```
    spot_id      INT PRIMARY KEY AUTO_INCREMENT,
    host_user_id INT NOT NULL,
    address_id   INT NOT NULL,
    spot_type_id INT NOT NULL,
    hourly_rate  DECIMAL(8,2) NOT NULL,
    is_active    BOOLEAN NOT NULL DEFAULT TRUE,
    instructions TEXT NULL,
    FOREIGN KEY (host_user_id) REFERENCES users(user_id),
    FOREIGN KEY (address_id)   REFERENCES addresses(address_id),
    FOREIGN KEY (spot_type_id) REFERENCES spot_types(spot_type_id)
);
```

4.1.8 spot_photos

```
CREATE TABLE spot_photos (
    photo_id    INT PRIMARY KEY AUTO_INCREMENT,
    spot_id     INT NOT NULL,
    photo_url   TEXT NOT NULL,
    uploaded_at DATETIME NOT NULL,
    FOREIGN KEY (spot_id) REFERENCES spots(spot_id)
);
```

4.1.9 availability_windows

```
CREATE TABLE availability_windows (
    window_id         INT PRIMARY KEY AUTO_INCREMENT,
    spot_id           INT NOT NULL,
    start_time        DATETIME NOT NULL,
    end_time          DATETIME NOT NULL,
    availability_kind VARCHAR(16) NOT NULL,
    FOREIGN KEY (spot_id) REFERENCES spots(spot_id)
);
```

4.1.10 reservation_statuses

```
CREATE TABLE reservation_statuses (
    status_id   INT PRIMARY KEY AUTO_INCREMENT,
    status_name VARCHAR(32) NOT NULL UNIQUE
);
```

4.1.11 reservations

```
CREATE TABLE reservations (
    reservation_id         INT PRIMARY KEY AUTO_INCREMENT,
    spot_id                INT NOT NULL,
    renter_user_id         INT NOT NULL,
    vehicle_id             INT NOT NULL,
    status_id              INT NOT NULL,
    start_time             DATETIME NOT NULL,
    end_time               DATETIME NOT NULL,
    hourly_rate_at_booking DECIMAL(8,2) NOT NULL,
    total_cost             DECIMAL(10,2) NOT NULL,
    created_at             DATETIME NOT NULL,
    FOREIGN KEY (spot_id)        REFERENCES spots(spot_id),
    FOREIGN KEY (renter_user_id) REFERENCES users(user_id),
    FOREIGN KEY (vehicle_id)     REFERENCES vehicles(vehicle_id),
    FOREIGN KEY (status_id)      REFERENCES reservation_statuses(status_id)
);
```

4.1.12 payments

```
CREATE TABLE payments (
    payment_id     INT PRIMARY KEY AUTO_INCREMENT,
    reservation_id INT NOT NULL UNIQUE,
    amount         DECIMAL(10,2) NOT NULL,
    paid_at        DATETIME NULL,
    method         VARCHAR(32) NULL,
    payment_status VARCHAR(32) NOT NULL,
    FOREIGN KEY (reservation_id) REFERENCES reservations(reservation_id)
);
```

4.1.13 reviews

```
CREATE TABLE reviews (
    reservation_id INT PRIMARY KEY,
    rating         TINYINT NOT NULL,
    comment        TEXT NULL,
    created_at     DATETIME NOT NULL,
    FOREIGN KEY (reservation_id) REFERENCES reservations(reservation_id)
);
```

4.2 Add Data (INSERT)

Representative INSERT statements for all tables. Full inserts are in parking_shark.sql.

Insert a new user:

```
INSERT INTO users (computing_id, first_name, last_name, email, created_at)
VALUES ('abc1de', 'Alice', 'Chen', 'ac@virginia.edu', NOW());
```

Insert a phone number for a user:

```
INSERT INTO user_phones (user_id, phone_number)
VALUES (1, '434-555-0101');
```

Insert a vehicle:

```
INSERT INTO vehicles (user_id, make, model, year, color)
VALUES (1, 'Toyota', 'Camry', 2020, 'Silver');
```

Insert a vehicle registration:

```
INSERT INTO vehicle_registrations
    (vehicle_id, license_plate, plate_state, expires_on, is_verified, verified_at)
VALUES (1, 'VA-ABC123', 'VA', '2026-12-31', TRUE, NOW());
```

Insert an address:

```
INSERT INTO addresses (street, city, state, zip_code)
VALUES ('100 Chancellor St', 'Charlottesville', 'VA', '22903');
```

Insert a spot:

```
INSERT INTO spots (host_user_id, address_id, spot_type_id, hourly_rate, is_active,
instructions)
VALUES (1, 1, 1, 5.00, TRUE, 'Pull into driveway on left. Do not block garage.');
```

Insert a spot photo:

```
INSERT INTO spot_photos (spot_id, photo_url, uploaded_at)
VALUES (1, 'https://parking-shark.s3.amazonaws.com/spots/1/photo1.jpg', NOW());
```

Insert an availability window:

```
INSERT INTO availability_windows (spot_id, start_time, end_time, availability_kind)
VALUES (1, '2025-10-01 08:00:00', '2025-10-01 18:00:00', 'Available');
```

Insert a reservation:

```
INSERT INTO reservations
    (spot_id, renter_user_id, vehicle_id, status_id,
     start_time, end_time, hourly_rate_at_booking, total_cost, created_at)
VALUES (1, 6, 6,
    (SELECT status_id FROM reservation_statuses WHERE status_name='Pending'),
    '2025-10-01 09:00:00', '2025-10-01 12:00:00', 5.00, 15.00, NOW());
```

Insert a payment:

```
INSERT INTO payments (reservation_id, amount, paid_at, method, payment_status)
VALUES (1, 15.00, NULL, 'Credit Card', 'Pending');
```

Insert a review (only for completed reservations):

```
INSERT INTO reviews (reservation_id, rating, comment, created_at)
VALUES (1, 5, 'Great spot! Very convenient location near UVA.', NOW());
```

4.3 Retrieve Data (SELECT)

Key SELECT queries used by the application.

Get all active spots with address and type (browse/search page):

```
SELECT s.spot_id, s.hourly_rate, s.instructions, a.street, a.city,
       st.type_name, u.first_name AS host_first, u.last_name AS host_last
FROM spots s
JOIN addresses a   ON s.address_id   = a.address_id
JOIN spot_types st ON s.spot_type_id = st.spot_type_id
JOIN users u       ON s.host_user_id = u.user_id
WHERE s.is_active = TRUE
ORDER BY s.hourly_rate ASC;
```

Get available spots for a given time window (search with conflict check):

```
SELECT s.spot_id, s.hourly_rate, a.street, a.city, st.type_name
FROM spots s
JOIN addresses a   ON s.address_id   = a.address_id
JOIN spot_types st ON s.spot_type_id = st.spot_type_id
WHERE s.is_active = TRUE
  AND EXISTS (
      SELECT 1 FROM availability_windows aw
      WHERE aw.spot_id = s.spot_id
        AND aw.availability_kind = 'Available'
        AND aw.start_time <= '2025-10-05 09:00:00'
        AND aw.end_time   >= '2025-10-05 12:00:00'
  )
  AND s.spot_id NOT IN (
      SELECT r.spot_id FROM reservations r
      JOIN reservation_statuses rs ON r.status_id = rs.status_id
      WHERE rs.status_name IN ('Pending','Confirmed')
        AND r.start_time < '2025-10-05 12:00:00'
        AND r.end_time   > '2025-10-05 09:00:00'
  );
```

Get all reservations for a renter (booking history):

```
SELECT r.reservation_id, r.start_time, r.end_time, r.total_cost,
       rs.status_name, a.street, a.city, v.make, v.model
FROM reservations r
JOIN reservation_statuses rs ON r.status_id  = rs.status_id
JOIN spots s                 ON r.spot_id     = s.spot_id
JOIN addresses a             ON s.address_id  = a.address_id
JOIN vehicles v              ON r.vehicle_id  = v.vehicle_id
WHERE r.renter_user_id = 6
ORDER BY r.start_time DESC;
```

Get all reservations on a host's spots (host dashboard):

```
SELECT r.reservation_id, r.start_time, r.end_time, r.total_cost,
       rs.status_name, u.first_name, u.last_name, a.street
FROM reservations r
JOIN reservation_statuses rs ON r.status_id      = rs.status_id
JOIN users u                 ON r.renter_user_id  = u.user_id
JOIN spots s                 ON r.spot_id          = s.spot_id
```

```
JOIN addresses a             ON s.address_id       = a.address_id
WHERE s.host_user_id = 1
ORDER BY r.start_time DESC;
```

Get average rating for a host (derived user_rating):

```
SELECT u.user_id, u.first_name, u.last_name,
       ROUND(AVG(rv.rating), 2) AS avg_rating,
       COUNT(rv.reservation_id) AS total_reviews
FROM users u
JOIN spots s        ON s.host_user_id    = u.user_id
JOIN reservations r ON r.spot_id         = s.spot_id
JOIN reviews rv     ON rv.reservation_id = r.reservation_id
WHERE u.user_id = 1
GROUP BY u.user_id, u.first_name, u.last_name;
```

Check for double-booking conflict before inserting:

```
SELECT COUNT(*) AS conflict_count
FROM reservations r
JOIN reservation_statuses rs ON r.status_id = rs.status_id
WHERE r.spot_id = 1
  AND rs.status_name IN ('Pending', 'Confirmed')
  AND r.start_time < '2025-10-01 14:00:00'
  AND r.end_time   > '2025-10-01 09:00:00';
```

Get all vehicles for a user:

```
SELECT v.vehicle_id, v.make, v.model, v.year, v.color,
       vr.license_plate, vr.plate_state, vr.is_verified
FROM vehicles v
LEFT JOIN vehicle_registrations vr ON v.vehicle_id = vr.vehicle_id
WHERE v.user_id = 6;
```

Get payment details for a reservation:

```
SELECT p.payment_id, p.amount, p.paid_at, p.method, p.payment_status
FROM payments p
WHERE p.reservation_id = 1;
```

Search spots by type and maximum price:

```
SELECT s.spot_id, s.hourly_rate, a.street, a.city, st.type_name
FROM spots s
JOIN addresses a   ON s.address_id   = a.address_id
JOIN spot_types st ON s.spot_type_id = st.spot_type_id
WHERE s.is_active = TRUE
  AND st.type_name = 'Driveway'
  AND s.hourly_rate <= 6.00
ORDER BY s.hourly_rate ASC;
```

Get spot details by spot_id (spot detail/listing page):

```
SELECT s.spot_id, s.hourly_rate, s.is_active, s.instructions,
       a.street, a.city, a.state, a.zip_code,
       st.type_name,
       u.user_id AS host_id, u.first_name AS host_first, u.last_name AS host_last
FROM spots s
JOIN addresses a   ON s.address_id   = a.address_id
JOIN spot_types st ON s.spot_type_id = st.spot_type_id
```

```
JOIN users u       ON s.host_user_id = u.user_id
WHERE s.spot_id = 1;
```

Get user profile by computing_id (login/profile page):

```
SELECT user_id, computing_id, first_name, last_name, email, created_at
FROM users
WHERE computing_id = 'abc1de';
```

Get all reviews for a specific spot (spot detail reviews section):

```
SELECT rv.reservation_id, rv.rating, rv.comment, rv.created_at,
       u.first_name AS renter_first, u.last_name AS renter_last
FROM reviews rv
JOIN reservations r ON rv.reservation_id = r.reservation_id
JOIN users u        ON r.renter_user_id  = u.user_id
WHERE r.spot_id = 1
ORDER BY rv.created_at DESC;
```

Get full reservation detail by reservation_id (booking confirmation page):

```
SELECT r.reservation_id, r.start_time, r.end_time, r.hourly_rate_at_booking,
r.total_cost,
       rs.status_name,
       s.spot_id, a.street, a.city, a.state, a.zip_code, st.type_name,
       u.first_name AS renter_first, u.last_name AS renter_last, u.email,
       v.make, v.model, v.year, v.color, vr.license_plate,
       p.amount, p.paid_at, p.method, p.payment_status
FROM reservations r
JOIN reservation_statuses rs ON r.status_id       = rs.status_id
JOIN spots s                 ON r.spot_id          = s.spot_id
JOIN addresses a             ON s.address_id       = a.address_id
JOIN spot_types st           ON s.spot_type_id     = st.spot_type_id
JOIN users u                 ON r.renter_user_id   = u.user_id
JOIN vehicles v              ON r.vehicle_id       = v.vehicle_id
LEFT JOIN vehicle_registrations vr ON v.vehicle_id  = vr.vehicle_id
LEFT JOIN payments p               ON r.reservation_id = p.reservation_id
WHERE r.reservation_id = 1;
```

4.4 Update Data (UPDATE)

Update spot details (host edits listing):

```
UPDATE spots
SET hourly_rate  = 5.50,
    instructions = 'Updated: Pull into driveway on left.',
    is_active    = TRUE
WHERE spot_id = 1 AND host_user_id = 1;
```

Change reservation status to Confirmed (host accepts booking):

```
UPDATE reservations
SET status_id = (SELECT status_id FROM reservation_statuses
                 WHERE status_name = 'Confirmed')
WHERE reservation_id = 7;
```

Change reservation status to Completed:

```
UPDATE reservations
SET status_id = (SELECT status_id FROM reservation_statuses
                 WHERE status_name = 'Completed')
WHERE reservation_id = 7;
```

Cancel a reservation:

```
UPDATE reservations
SET status_id = (SELECT status_id FROM reservation_statuses
                 WHERE status_name = 'Cancelled')
WHERE reservation_id = 9;
```

Mark payment as paid:

```
UPDATE payments
SET paid_at = NOW(), payment_status = 'Paid'
WHERE reservation_id = 4;
```

Update user profile information:

```
UPDATE users
SET first_name = 'Alicia', email = 'alicia.chen@virginia.edu'
WHERE user_id = 1;
```

Mark vehicle registration as verified:

```
UPDATE vehicle_registrations
SET is_verified = TRUE, verified_at = NOW()
WHERE vehicle_id = 3;
```

Deactivate a spot:

```
UPDATE spots SET is_active = FALSE
WHERE spot_id = 6 AND host_user_id = 3;
```

Update a review comment (renter edits their review):

```
UPDATE reviews
SET rating  = 5,
    comment = 'Spot was perfect — easy to find and exactly as described!'
WHERE reservation_id = 1;
```

4.5 Delete Data (DELETE)

Delete a spot photo:

```
DELETE FROM spot_photos
WHERE photo_id = 12 AND spot_id = 15;
```

Delete an availability window:

```
DELETE FROM availability_windows
WHERE window_id = 3 AND spot_id = 1;
```

Remove a user's phone number:

```
DELETE FROM user_phones
WHERE user_id = 2 AND phone_number = '434-555-0199';
```

Delete a vehicle registration record (host/admin removes registration):

```
DELETE FROM vehicle_registrations
WHERE vehicle_id = 3;
```

Delete a review (admin or host removes inappropriate review):

```
DELETE FROM reviews
WHERE reservation_id = 5;
```

Delete a cancelled reservation record (data cleanup):

```
DELETE FROM reservations
WHERE reservation_id = 9
  AND status_id = (SELECT status_id FROM reservation_statuses
                   WHERE status_name = 'Cancelled');
```

Delete a vehicle safely using derived table (MariaDB compatible):

```
DELETE FROM vehicles
WHERE vehicle_id = 3
  AND vehicle_id NOT IN (
      SELECT vehicle_id FROM (
          SELECT vehicle_id FROM reservations
      ) AS reserved
  );
```

4.6 Advanced SQL Commands

The Parking Shark system implements three different advanced SQL command types: a TRIGGER, a STORED PROCEDURE, and CHECK CONSTRAINTs. This satisfies the requirement of at least two different advanced SQL command types (four members = 2 required; three are provided for completeness).

4.6.1 TRIGGER — Prevent Double-Booking

This BEFORE INSERT trigger fires whenever a new row is added to the reservations table. It checks whether the target spot already has a Pending or Confirmed reservation that overlaps the requested time window. If a conflict is detected, SIGNAL raises an application-level error and the insert is aborted, ensuring no double-booking can occur even if the application layer fails to check first. Note: if an unrecognized status_id is inserted (not present in reservation_statuses), the SELECT INTO returns NULL, the IF condition evaluates to FALSE, and the conflict check is silently skipped — this is an intentional design choice, as only valid Pending/Confirmed statuses should trigger the check.

```
DELIMITER //

CREATE TRIGGER prevent_double_booking
BEFORE INSERT ON reservations
FOR EACH ROW
BEGIN
    DECLARE conflict_count INT;
    DECLARE new_status_name VARCHAR(32);

    SELECT status_name INTO new_status_name
    FROM reservation_statuses
    WHERE status_id = NEW.status_id;

    IF new_status_name IN ('Pending', 'Confirmed') THEN
        SELECT COUNT(*) INTO conflict_count
        FROM reservations r
        JOIN reservation_statuses rs ON r.status_id = rs.status_id
        WHERE r.spot_id      = NEW.spot_id
          AND rs.status_name IN ('Pending', 'Confirmed')
          AND r.start_time   < NEW.end_time
          AND r.end_time     > NEW.start_time;

        IF conflict_count > 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Booking conflict: spot is already reserved for this time.';
        END IF;
    END IF;
END //

DELIMITER ;
```

4.6.2 STORED PROCEDURE — create_booking

This procedure encapsulates the complete booking workflow in a single atomic call: it looks up the spot's current rate, calculates the total cost, checks for conflicts, inserts the reservation, and creates the corresponding payment record. A START TRANSACTION / COMMIT / ROLLBACK block wraps the two INSERT statements, ensuring both succeed or fail together and preventing orphaned reservation records if the payment INSERT fails.

```
DELIMITER //

CREATE PROCEDURE create_booking(
    IN  p_spot_id        INT,
    IN  p_renter_user_id INT,
    IN  p_vehicle_id     INT,
    IN  p_start_time     DATETIME,
    IN  p_end_time       DATETIME,
    IN  p_payment_method VARCHAR(32),
    OUT p_reservation_id INT
)
BEGIN
    DECLARE v_hourly_rate   DECIMAL(8,2);
    DECLARE v_total_cost    DECIMAL(10,2);
    DECLARE v_pending_id    INT;
    DECLARE v_conflict_cnt  INT;
    DECLARE v_hours         DECIMAL(10,4);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Step 1: Get the spot's hourly rate
    SELECT hourly_rate INTO v_hourly_rate
    FROM spots
    WHERE spot_id = p_spot_id AND is_active = TRUE;

    IF v_hourly_rate IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Spot not found or is not active.';
    END IF;

    -- Step 2: Calculate cost (in fractional hours)
    SET v_hours      = TIMESTAMPDIFF(MINUTE, p_start_time, p_end_time) / 60.0;
    SET v_total_cost = ROUND(v_hourly_rate * v_hours, 2);

    -- Step 3: Check for conflicts
    SELECT COUNT(*) INTO v_conflict_cnt
    FROM reservations r
    JOIN reservation_statuses rs ON r.status_id = rs.status_id
    WHERE r.spot_id      = p_spot_id
```

```
      AND rs.status_name IN ('Pending', 'Confirmed')
      AND r.start_time   < p_end_time
      AND r.end_time     > p_start_time;

    IF v_conflict_cnt > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking conflict: the spot is unavailable for the
requested time.';
    END IF;

    -- Step 4: Get 'Pending' status id
    SELECT status_id INTO v_pending_id
    FROM reservation_statuses
    WHERE status_name = 'Pending';

    -- Step 5: Insert reservation
    INSERT INTO reservations
        (spot_id, renter_user_id, vehicle_id, status_id,
         start_time, end_time, hourly_rate_at_booking, total_cost, created_at)
    VALUES
        (p_spot_id, p_renter_user_id, p_vehicle_id, v_pending_id,
         p_start_time, p_end_time, v_hourly_rate, v_total_cost, NOW());

    SET p_reservation_id = LAST_INSERT_ID();

    -- Step 6: Create payment record
    INSERT INTO payments (reservation_id, amount, paid_at, method, payment_status)
    VALUES (p_reservation_id, v_total_cost, NULL, p_payment_method, 'Pending');

    COMMIT;
END //
DELIMITER ;

-- Example usage:
-- CALL create_booking(2, 6, 6,
--     '2025-11-01 09:00:00', '2025-11-01 12:00:00',
--     'Credit Card', @new_id);
-- SELECT @new_id;
```

4.6.3 CHECK CONSTRAINTs — Domain Integrity

CHECK constraints enforce valid domain values at the database engine level, independent of application code. The following five constraints are added after table creation to ensure data correctness:

Rating must be between 1 and 5:

```
ALTER TABLE reviews
ADD CONSTRAINT chk_rating_range
CHECK (rating BETWEEN 1 AND 5);
```

Hourly rate must be positive:

```
ALTER TABLE spots
ADD CONSTRAINT chk_hourly_rate_positive
```

```
CHECK (hourly_rate > 0);
```

Payment status must be one of the allowed values:

```
ALTER TABLE payments
ADD CONSTRAINT chk_payment_status_values
CHECK (payment_status IN ('Pending', 'Paid', 'Refunded', 'Failed'));
```

Availability kind must be 'Available' or 'Blocked':

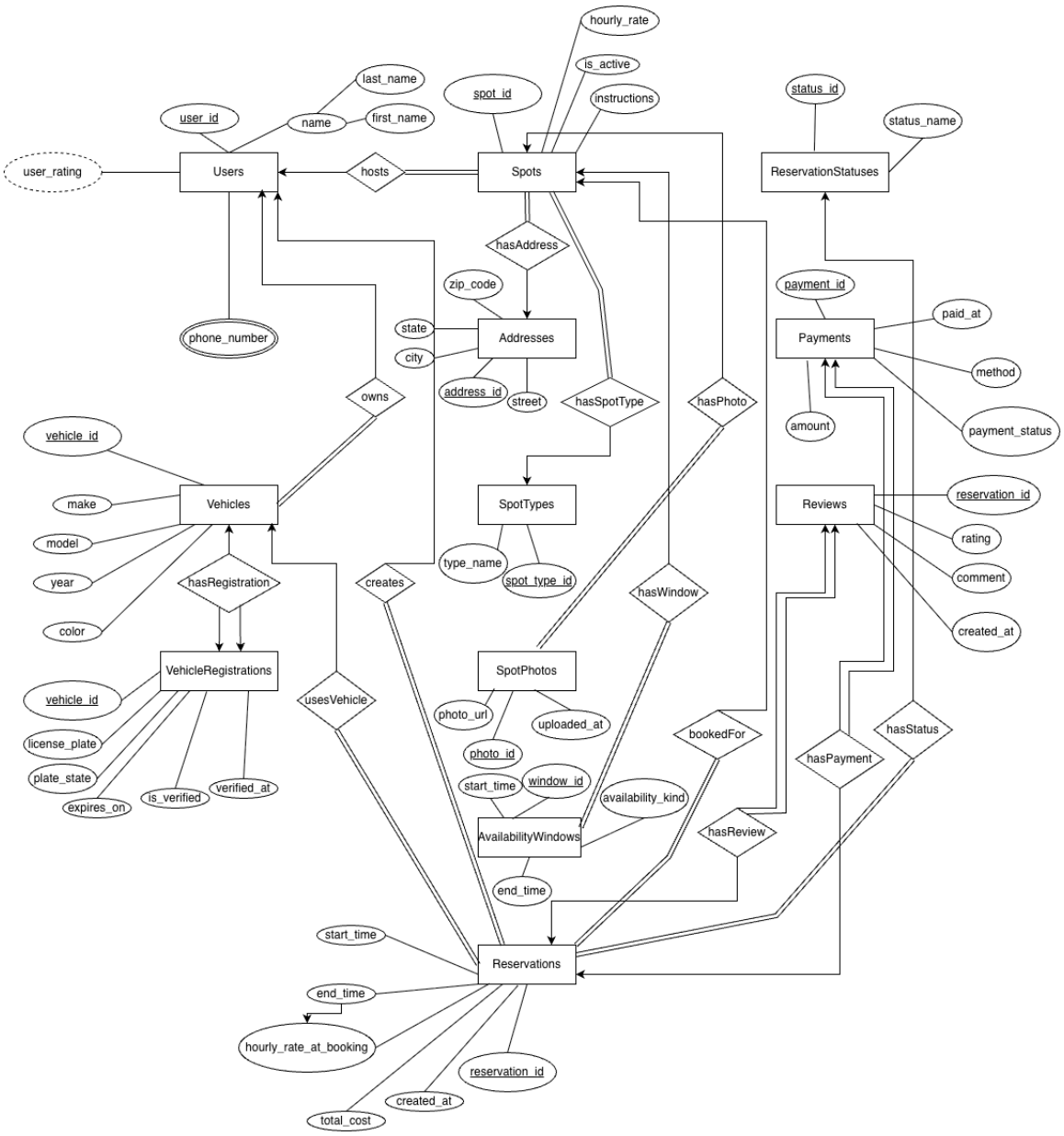
```
ALTER TABLE availability_windows
ADD CONSTRAINT chk_availability_kind_values
CHECK (availability_kind IN ('Available', 'Blocked'));
```

Reservation end_time must be after start_time:

```
ALTER TABLE reservations
ADD CONSTRAINT chk_reservation_time_order
CHECK (end_time > start_time);
```

All advanced SQL commands have been tested against the live database. The trigger, stored procedure, and check constraints are all active in the database and enforce the business logic described above.

Final ER Diagram (Same as Milestone 1)


hourly_rate
is_active
last_name
spot_id
instructions
status_id
user_id
name
first_name
status_name
user_rating
Users
hosts
Spots
ReservationStatuses
hasAddress
zip_code
payment_id
phone_number
state
Addresses
Payments
paid_at
city
method
owns
address_id
street
hasSpotType
hasPhoto
amount
payment_status
vehicle_id
make
Vehicles
SpotTypes
Reviews
reservation_id
model
rating
type_name
spot_type_id
year
hasRegistration
creates
hasWindow
comment
color
created_at
VehicleRegistrations
SpotPhotos
vehicle_id
usesVehicle
photo_url
uploaded_at
bookedFor
hasStatus
license_plate
photo_id
hasPayment
plate_state
start_time
window_id
availability_kind
expires_on
is_verified
verified_at
AvailabilityWindows
hasReview
end_time
start_time
Reservations
end_time
hourly_rate_at_booking
reservation_id
created_at
total_cost